Blocks, fixed effects, and repeated measurements

Lecture notes

Foundations of Regression and Predictive Modeling

Table of contents

Nuisance variation 1
Case 1: the kiosk upsell prompt 2
The naive model and where its noise comes from 3
Fixed effects: the blocked analysis 3
A second nuisance factor, adjusted rather than blocked 4
Case 2: the warehouse scanner study 5
Paired analysis three ways 6
Counterbalancing and the order effect 7
The assignment unit: why the ticket log lies 8
Fixed effects or random effects? 8
Summary 9

Nuisance variation

The last module analyzed experiments whose units were interchangeable: one past-due account looked much like another, one customer like another. Real experiments usually run inside structure. Treatments get assigned within stores, batches, clinics, shifts, or teams, and sometimes the same unit is measured more than once.

That structure creates **nuisance variation**: variation in the outcome that is real and often large, but that has nothing to do with the question. A downtown taco shop rings up bigger tickets than a suburban one whether or not the kiosk suggests a drink. A fast warehouse picker outpaces a slow one on any scanner. Left alone, nuisance variation lands in the regression's error term, inflates the standard error, and can bury a treatment effect entirely.

This module is about the standard tools for getting nuisance variation out of the way: blocking in the design, fixed effects in the model, and paired or repeated measurements as the strongest version of both.

Case 1: the kiosk upsell prompt

A Texas fast-casual taco chain wants to know whether a kiosk checkout prompt — "Add a large agua fresca for $1.50?" — raises the average ticket. The prompt is a store-wide kiosk configuration, so individual customers cannot be randomized. The randomization unit is the **store-day**: for each of 16 stores over 30 days, each day is assigned prompt-on or prompt-off, with exactly 15 of each per store.

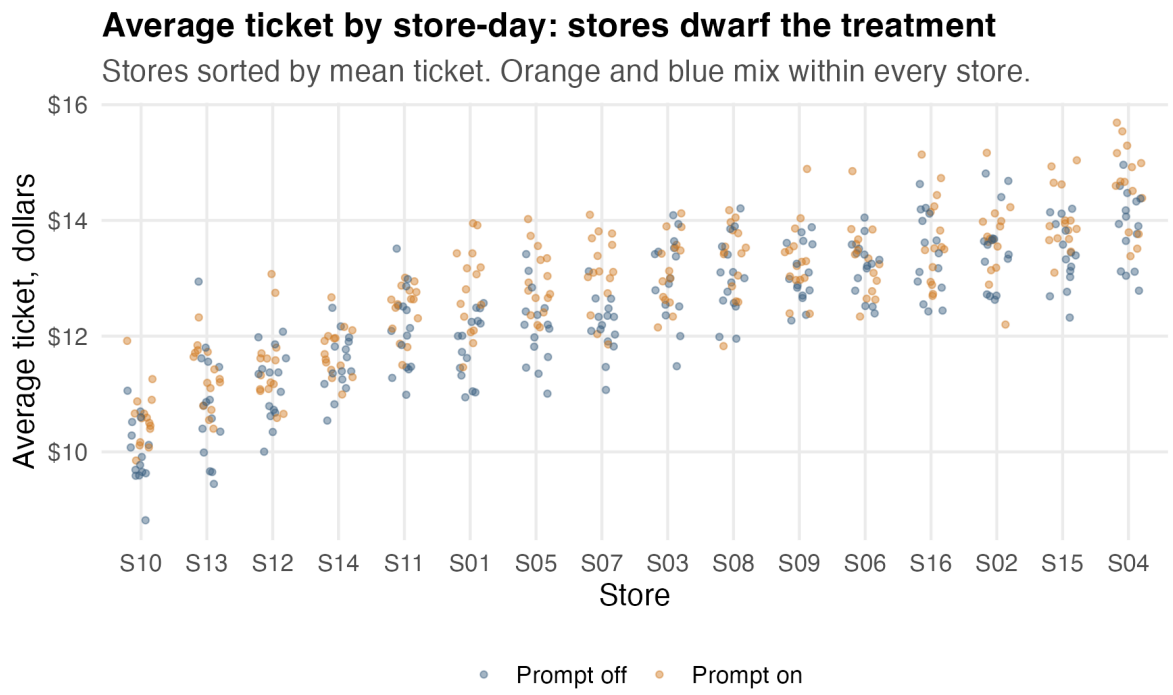
That "exactly 15 of each per store" is the design decision that matters. Each store serves as a **block**: a group of more comparable units (days at the same store) inside which the experiment is run separately.

```
upsell <- read_csv(file.path(module_dir, "upsell_prompt.csv")) |>
  mutate(day_of_week = factor(day_of_week,
    levels = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
  ))

upsell |> head(4)
```

```
# A tibble: 4 x 5
  store_id   day day_of_week prompt avg_ticket
  <chr>    <dbl> <fct>        <dbl>      <dbl>
1 S01          1 Mon              0       11.6
2 S01          2 Tue              1       12.1
3 S01          3 Wed              1       13.2
4 S01          4 Thu              0       11.4
```

The stores differ enormously. Store-level mean tickets run from about $10.30 to $14.20 — a four-dollar spread, when the effect being hunted is a few dimes:



The naive model and where its noise comes from

Ignore the stores and fit the one-dummy model from last module:

```
naive_model <- lm(avg_ticket ~ prompt, data = upsell)
tidy(naive_model, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic   p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1 (Intercept)   12.4      0.0790    157.    0           12.2      12.5
2 prompt         0.451    0.112       4.04 0.0000632    0.231     0.670
```

The estimate, about $0.45, is fine: because the randomization was balanced within every store, the naive comparison is not biased. But the 95% interval runs from $0.23 to $0.67 — too wide to price a chain-wide rollout.

The width comes from the model's single error bucket. Into it went day-to-day noise within stores *and* the entire four-dollar spread between stores. The between-store spread is nuisance: real, large, and irrelevant to the prompt.

Fixed effects: the blocked analysis

Since randomization happened within stores, the comparison should too: on prompt days, was each store above *its own* normal? In regression form, that is store dummies:

```r
store_fe_model <- lm(avg_ticket ~ prompt + store_id, data = upsell)
tidy(store_fe_model, conf.int = TRUE) |> filter(term == "prompt")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic  p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 prompt    0.451    0.0579      7.78 4.76e-14    0.337     0.565
```

The point estimate is unchanged to the fourth decimal — with a balanced design, blocking has nothing to fix about the estimate. The interval, though, is nearly half as wide: about \$0.34 to \$0.57. Randomization protects against bias; blocking attacks noise. Those are different jobs, and this is the cleanest demonstration of the difference.

The 15 store coefficients are **fixed effects**: dummies included to absorb stable group differences, never interpreted individually. The same device handles batches, evaluators, clinics, machines, and months.

A second nuisance factor, adjusted rather than blocked

Weekends run about \$0.80 hotter than weekdays, and the coin flips did not block on day of week — they only balanced it on average:

```r
store_dow_model <- lm(avg_ticket ~ prompt + store_id + day_of_week, data = upsell)
tidy(store_dow_model, conf.int = TRUE) |> filter(term == "prompt")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic  p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 prompt    0.400    0.0491      8.15 3.63e-15    0.303     0.496
```

Two things happened. The estimate moved a little (to about \$0.40), because by chance the prompt-on days landed slightly unevenly across weekdays, and the adjustment corrected for it. And the interval tightened further, because day-of-week variation left the error bucket.

This is the difference between a **blocked design** and a **post-hoc adjusted analysis**. Store was in the design: 15/15 within every store, guaranteed by construction. Day of week was only in the model: balanced by luck, corrected afterward, and dependent on our having measured it. Both use dummies; they are not equally strong. The working rule:

> Block what you can. Model what you must. Caveat what you can neither control nor measure.

Case 2: the warehouse scanner study

A grocery distributor's warehouse in San Marcos is considering new ergonomic RF scanners at $1,100 each, 300 units. The pilot uses 40 pickers. Each picker works one week on the old scanner and one week on the new one, with the order decided by a coin flip per picker — 20 start with each device. That ordering trick is **counterbalancing**.

```
scanner <- read_csv(file.path(module_dir, "scanner_study.csv")) |>
  mutate(device = factor(device, levels = c("old", "new")))

scanner |> arrange(picker_id) |> head(4)
```

```
# A tibble: 4 x 4
  picker_id  week device picks_per_hour
  <chr>     <dbl> <fct>           <dbl>
1 P01           1 old              67
2 P01           2 new              80.4
3 P02           1 new              90.7
4 P02           2 old              88.3
```

The naive analysis treats the 80 picker-weeks as two independent groups:

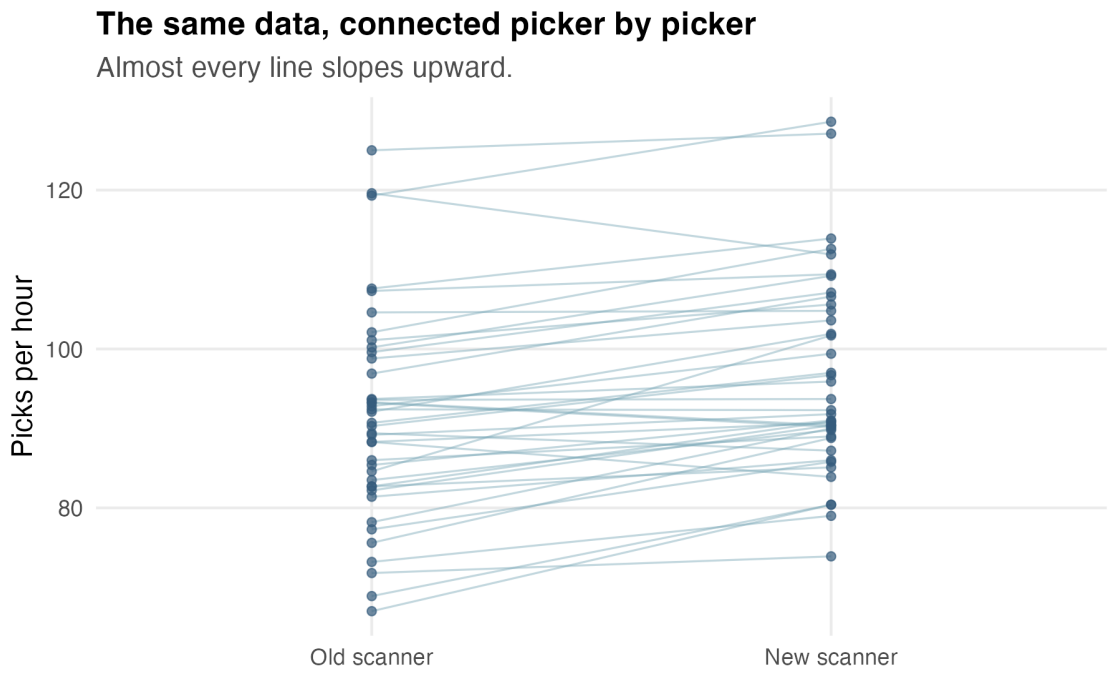
```
naive_scanner <- lm(picks_per_hour ~ device, data = scanner)
tidy(naive_scanner, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)    91.2       2.03     45.1  1.33e-57   87.2        95.3
2 devicenew       5.09      2.86      1.78 7.92e- 2   -0.607      10.8
```

New minus old is about +5.1 picks per hour, but the interval spans −0.6 to +10.8 and $p = 0.08$. Verdict: no detectable effect.

The naive verdict is wrong, and the reason is instructive. Pickers range from about 70 to about 125 picks per hour, so the two clouds overlap almost completely. All of that picker-to-picker spread sits in the error term. But the design measured *every picker on both devices*, and the naive model never used that fact.

Connect each picker's two measurements and the picture changes entirely:



34 of 40 lines slope upward. The effect was there all along, hiding behind nuisance variation.

Paired analysis three ways

The classical tool is the paired t-test on within-picker differences:

```
diffs <- scanner |>
  select(picker_id, device, picks_per_hour) |>
  pivot_wider(names_from = device, values_from = picks_per_hour) |>
  mutate(diff = new - old)

t.test(diffs$diff)
```

```
    One Sample t-test

data:  diffs$diff
t = 6.0966, df = 39, p-value = 3.803e-07
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 3.404603 6.785397
sample estimates:
mean of x
    5.095
```

The regression version replaces the differencing with 39 picker dummies:

```r
picker_fe_model <- lm(picks_per_hour ~ device + picker_id, data = scanner)
tidy(picker_fe_model, conf.int = TRUE) |> filter(term == "devicenew")
```

```
# A tibble: 1 x 7
  term      estimate std.error statistic     p.value conf.low conf.high
  <chr>        <dbl>     <dbl>     <dbl>       <dbl>    <dbl>     <dbl>
1 devicenew     5.09     0.836      6.10 0.000000380     3.40      6.79
```

Identical estimate, identical standard error, identical t statistic. A paired t-test *is* a fixed-effects regression with blocks of size two. The device effect is $+5.09$ picks per hour with interval $[3.40, 6.79]$ — the same point estimate as the naive model, but the standard error fell from 2.86 to 0.84, because everything stable about a person cancels out exactly when the comparison never crosses a person.

Pairing is blocking pushed to its limit, and it is why within-subject designs punch so far above their sample size. Forty pickers settled a $330K hardware question.

Counterbalancing and the order effect

Week 2 came with a learning bump — everyone knew the routes a little better. Because the device order was randomized, that bump split evenly across devices instead of contaminating the device effect, and it can be modeled directly:

```r
picker_week_model <- lm(
  picks_per_hour ~ device + factor(week) + picker_id,
  data = scanner
)
tidy(picker_week_model, conf.int = TRUE) |>
  filter(!str_detect(term, "picker_id"))
```

```
# A tibble: 3 x 7
  term          estimate std.error statistic  p.value conf.low conf.high
  <chr>            <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)      70.4      2.61      26.9  2.38e-26  65.1        75.7
2 devicenew         5.09     0.807      6.31 2.11e- 7   3.46        6.73
3 factor(week)2     1.58     0.807      1.96 5.76e- 2  -0.0535      3.21
```

The week-2 effect is about $+1.6$ picks per hour; the device effect barely moves. If every picker had used the new scanner second, the learning bump would have been inseparable from the device effect. Whenever units are measured more than once, randomize the order.

The assignment unit: why the ticket log lies

Every store-day in the upsell experiment summarizes about 900 individual tickets, so the full ticket log holds roughly 432,000 rows. The temptation is obvious: more rows, smaller standard errors. Zooming into a single store makes the trap visible. Store S07 has 30 days and about 27,000 tickets, and the same comparison can be run two ways:

```
tickets <- read_csv(file.path(module_dir, "upsell_tickets_s07.csv"))

ticket_level <- lm(ticket_amount ~ prompt, data = tickets)
tidy(ticket_level, conf.int = TRUE) |> filter(term == "prompt")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic  p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 prompt    0.939    0.0853      11.0 3.96e-28    0.772      1.11
```

```
day_level <- tickets |>
  group_by(day, prompt) |>
  summarize(avg_ticket = mean(ticket_amount), .groups = "drop") |>
  lm(avg_ticket ~ prompt, data = _)
tidy(day_level, conf.int = TRUE) |> filter(term == "prompt")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic  p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 prompt    0.939     0.212      4.43 0.000132    0.505      1.37
```

The two analyses give the same estimate to four decimal places, but the 27,000-row interval is about 2.5 times narrower than the 30-row interval — and it is the narrow one that is wrong. The coin was flipped once per store-day, not once per ticket. All 900 tickets in a day share that one flip, along with the day's weather, staffing, and foot traffic, so they are not independent pieces of evidence about the prompt. Treating them as independent manufactures precision that does not exist; every p-value computed that way is overconfident.

The rule is the one the experiments module stated and this example prices: **analyze at the level you randomized**. Aggregate rows to the randomized unit, as the store-day means did all along, or use a model that explicitly respects the grouping — which is exactly what the mixed models of the hierarchical module do. The store-day means were never a convenience; they were the correct unit.

Fixed effects or random effects?

The hierarchical-models module handled grouped data with partial pooling, so which tool goes with which problem?

Use **fixed effects** when there are a handful to a few hundred groups, reasonable data per group, and the groups are pure nuisance — you want them out of the way and will never report them. Use **partial pooling** (`stan_lmer`, `stan_glmer`) when there are many groups, sparse or unbalanced data per group, or the group-level estimates are themselves a deliverable and sparse groups need shrinkage toward the population average.

The scanner study — 40 pickers, two balanced measurements each, nobody cares about individual pickers — is squarely a fixed-effects problem. The emergency-department physicians from the hierarchical module, with wildly uneven caseloads and per-physician estimates as the goal, were squarely a partial-pooling problem.

Summary

- Nuisance variation is real variation the question did not ask about. Untreated, it inflates standard errors and can bury an effect, as it did in the naive scanner analysis.
- Blocking randomizes within groups and compares within groups. It improves precision; randomization was already handling bias.
- Fixed effects are the regression form of blocking: many dummies for a nuisance grouping, included but never interpreted.
- A blocked factor is balanced by construction; an adjusted factor is balanced by chance and corrected by the model. Block what you can, model what you must, caveat the rest.
- Paired designs and repeated measurements make each unit its own control. A paired t-test is a fixed-effects regression with blocks of size two.
- Counterbalance the order of repeated measurements so time effects split across treatments.
- Choose fixed effects for a modest number of nuisance groups with decent data; choose partial pooling when groups are many, sparse, or themselves the object of interest.